

บทที่ 3

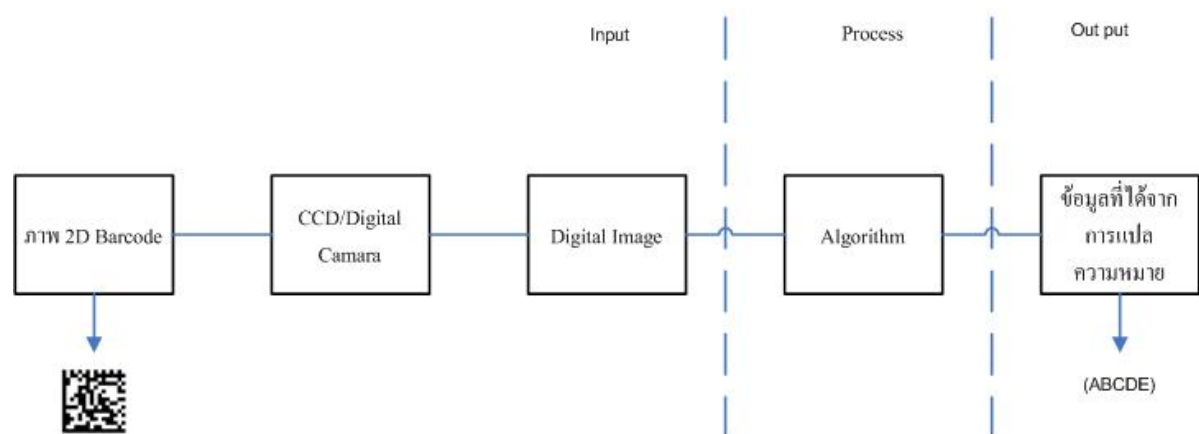
หลักการในการสร้างและออกแบบ

3.1 บทนำ

ในโครงการงานการจำแนกรหัสแท่งด้วยการประมวลผลภาพนี้ได้นำแนวคิดและหลักการทาง Image processing มาใช้ โดยการนำมาประยุกต์ใช้บนโทรศัพท์มือถือที่มีกล้องดิจิทัลและสนับสนุนแพลตฟอร์มของ J2ME ในส่วนของบทนี้จะได้ทำการนำเสนอการออกแบบระบบการจำแนกรหัสแท่งด้วยการประมวลผลภาพโดยทั่วไป

3.2 โครงสร้างและการทำงานของระบบ

ภาพรวมของระบบการประมวลผลภาพโดยจะมีการทำงานเป็นส่วนๆตามรูปที่ 3.1



รูปที่ 3.1 ภาพรวมของระบบประมวลผลภาพ

3.3 การออกแบบระบบในส่วนของการประมวลผลภาพ

3.3.1 การทำภาพไบนารี (Binarization)

ในการประมวลผลภาพนั้น ในขั้นตอนแรกจะเป็นการเตรียมข้อมูลภาพที่จะนำมาใช้ในการประมวลผลเพื่อให้ข้อมูลภาพมีรูปแบบที่เหมาะสมในการประมวลผล ดังนั้นสิ่งที่จำเป็นอย่างมากก็คือ การทำ Binarization หรือว่าการทำให้รูปเป็นแค่รูปขาวหรือดำเท่านั้น กล่าวคือทำให้ค่าของแต่ละพิกเซล เป็น 0 หรือ 1 ไม่ใช่สีแบบ 1 channel ซึ่งอาจจะมีได้ตั้งแต่ 0 -255 โดยค่าของ

จุดภาพใดๆที่มีค่าน้อยกว่าค่าเทรชโฮลด์ (Threshold) จะถูกกำหนดให้เป็น 0 (จุดดำ) และถ้าค่าของจุดภาพใด ๆ ที่มีค่ามากกว่า หรือเท่ากับค่า Threshold ก็จะถูกเปลี่ยนให้เป็น 1 (จุดขาว)

โดยค่าในการแปลงภาพสีไปเป็นภาพขาวหรือดำนั้นทำได้โดยการหาค่า Threshold ที่เหมาะสมที่สุดในการแปลงภาพสีให้เป็นภาพขาวหรือดำ (Binary Image) ในส่วนของโครงงานนี้จะใช้วิธีการคำนวณหา threshold โดยการกำหนดค่ากลาง ค่าดังกล่าวคือ การหาค่าเฉลี่ยของทุก พิกเซลในภาพนั่นเอง



รูปที่3.2 รูปของภาพจากการถ่ายด้วยโทรศัพท์มือถือ

สิ่งแรกที่ต้องทำก็คือ การทำให้ภาพสีจากด้านบนเป็นรูป Gray Scale ก่อน ซึ่งก็จะทำให้แต่ละ pixel ของภาพมีค่าตั้งแต่ 0-255 (ดำ ... เทาเข้ม ... เทาอ่อน ... ขาว)

จากนั้นก็ทำการ Low pass Filter ซึ่งเป็นวิธีการทำให้ภาพเบลอ และเป็นการลดสัญญาณรบกวนภายในภาพด้วย หลักของการใช้การกรองแบบเฉลี่ย คือ การลดรายละเอียดที่ไม่ตรงประเด็นในภาพต้นแบบออกไป กระบวนการนี้จะเอาค่าบริเวณรอบๆ พิกเซลมาคำนวณ แล้วนำมาแทนค่าพิกเซลเดิม บริเวณ Pixel รอบๆนี้เรียกว่า Mask ส่วนในการคำนวณบริเวณรอบๆเรียกว่าการทำ Convolution Mask ในที่นี้จะใช้เมตริก 3x3 เป็นหลักโดยใช้ค่าดังต่อไปนี้

ค่าพิกเซลต้นฉบับ					Mask ขนาด 3x3			ค่าที่ได้จากการคำนวณ				
12	0	20	20	23	X	$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$	=	?	?	?
12	10	5	123	56		$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$		?	11	?
9	20	11	99	34		$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$		?	?	?
20	10	123	67	77		$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$				
20	10	20	45	87		$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$				

$$\text{ค่าพิกเซลใหม่} = (12+0+20+12+10+5+9+20+11)/9$$

$$= 11$$

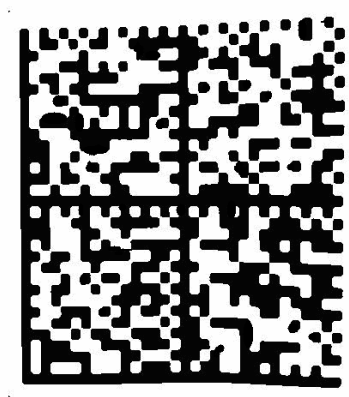
รูปที่ 3.3 แสดงการทำ Low pass Filter ที่ Mask ขนาด 3×3

การทำ Low pass Filter ที่ Mask ขนาด 3×3 ดังกล่าว จะเห็นว่าแถวแรกและแถวสุดท้ายกับคอลัมน์ซ้ายสุดและขวาสุดจะไม่สามารถคำนวณได้ ส่วนวิธีการแก้ไขแถวและคอลัมน์ที่หายไปโดยส่วนใหญ่ จะเอาแถวและคอลัมน์ตรงข้ามมาคำนวณ เช่น แถวบนสุดก็จะเอาแถวล่างสุดมาคำนวณ พิกเซลแถวแรก

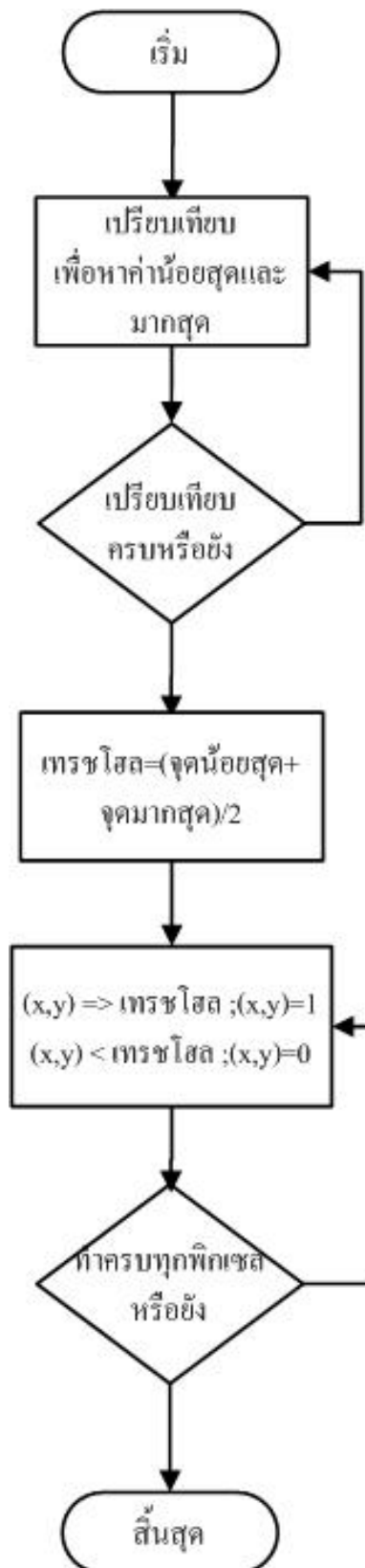


รูปที่ 3.4 รูปที่ผ่านการทำ Low pass Filter ที่ Mask ขนาด 3×3

จากนั้นก็ทำการหาค่า Threshold ที่จะใช้ในการทำ Binarization โดยการหาค่าเฉลี่ยของทุกๆ พิกเซลในภาพนั่นเอง ซึ่งจะแสดงผลได้ดังรูปด้านล่าง



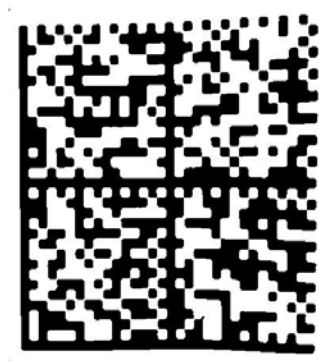
รูปที่ 3.5 รูปที่ผ่านการลดสัญญาณรบกวนและการทำ Threshold



รูปที่3.6 ขั้นตอนในการทำ Threshold

จากรูปที่3.5 จะเห็นได้ว่าสภาพแสงของภาพดั้งเดิมนั้นค่อนข้างมืดซึ่งเมื่อหลังจากการทำให้เป็น Gray Scale และหาค่า Mean ของพิกเซลทุกพิกเซลในภาพเพื่อนำไปใช้เป็น Threshold ในการทำ Binarization นั้น มีค่าน้อย ซึ่งก็ทำให้หลังจากการทำ Binarization แล้วจะได้จะทำให้ภาพรหัสแท่งที่ได้ไม่เห็นส่วนที่เป็นข้อมูลกล่าวคือภาพที่ได้ไม่มีความสมบูรณ์พอที่จะนำมาทำการแปลความหมายดังนั้นสิ่งที่จำเป็นต้องทำคือการปรับปรุงวิธีการหาค่า Threshold ขึ้นมาใหม่โดยในที่นี้เราจะใช้ Adaptive Threshold

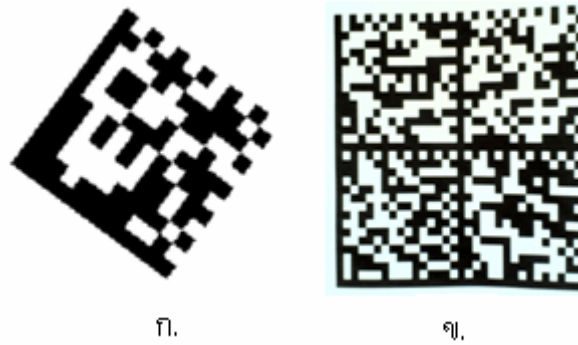
การทำ Binarization โดยใช้ Adaptive Threshold ก็คือการทำ Binarization โดยใช้ค่า Threshold ที่เปลี่ยนค่าได้ โดยไม่ใช่ค่าเดียวกันทั้งรูป ซึ่งวิธีนี้ก็จะทำให้เราสามารถลดอัตราความแตกต่างกันของแสงได้ดี เป็นการชดเชยแสงที่เปลี่ยนไปในสภาวะแวดล้อมต่างๆโดยค่าของ Threshold ก็ยังคงใช้เป็นค่า Mean ของพิกเซลในรูป แต่ว่าไม่ได้ใช้ Mean ของพิกเซลทุกพิกเซลในรูป แต่ว่าใช้เพียงพิกเซลที่อยู่รอบๆใกล้ๆกับบริเวณพิกเซลที่เราจะเลือกให้มันเป็นสีดำ 0 หรือสีขาว 1 โดยขนาดของกรอบพิกเซลข้างๆนั้นก็แล้วแต่เราพิจารณาตนเอง แต่ในที่นี้จะใช้ประมาณ 5% ของความสูงภาพ ซึ่งก็ทำให้ได้ผลที่ดีมากดังรูปด้านล่าง



รูปที่3.7 รูปที่ผ่านการทำ Binarization โดยการใช้ค่า Adaptive Threshold

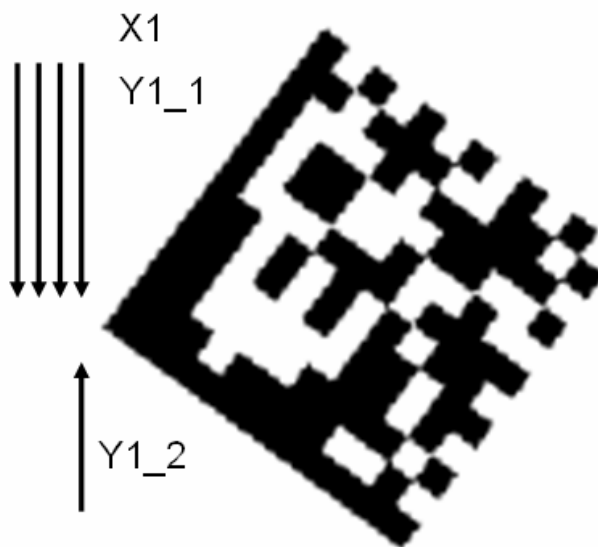
3.3.2 การปรับปรุงภาพที่ไม่สมบูรณ์ในกรณีต่างๆ

การปรับปรุงภาพที่ไม่สมบูรณ์ในกรณีต่างๆ เช่น ภาพเอียง พื้นผิวโค้ง บิดเบี้ยว ไม่สม่ำเสมอ แสงไม่เพียงพอ มีดเก็นไป สว่างเกินไป เพื่อให้ภาพที่ได้เหมาะสมในการประมวลผลเป็นสิ่งที่มีความจำเป็นมาก จึงต้องมีการวิเคราะห์และปรับปรุงรูปแบบในการรู้จำของระบบให้มีประสิทธิภาพยิ่งขึ้น



รูปที่3.8 รูปรหัสแท่ง 2 บิตที่มีลักษณะไม่สมบูรณ์ในกรณีต่างๆ

ในส่วนของการจัดการกับข้อมูลภาพรหัสแท่ง 2 บิตในลักษณะของภาพถ่ายที่ได้นั้น เกิดความไม่สมบูรณ์ในกรณีที่มุมของรูปภาพเกิดการเอียงดังรูปที่ 3.3 ก เราได้ใช้หลักการในการจัดการดังต่อไปนี้

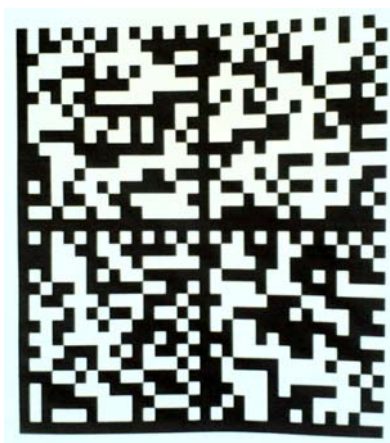


รูปที่3.9 แสดงวิธีการหมุนภาพ

เริ่มจากสแกนหามุมภาพ ทั้ง 4 มุม การหามุมจะต้องหาด้านที่เป็นแนวสแกนไว้ทั้ง 2 ด้าน แล้วนำมาเปรียบเทียบเพื่อให้ได้ค่าที่น่าจะเป็นค่าที่ใกล้เคียงกับมุมที่สุด เมื่อ ได้ มุมทั้ง 4 แล้ว นำมุม

ทั้ง 4 ที่ได้มาหาองศา โดยมุมที่ใกล้ 90 องศามากที่สุดจะต้องเป็นมุมที่อยู่ล่าง แล้วทำการหมุนภาพตามทฤษฎีที่ได้กล่าวไปแล้ว

การจัดการกับข้อมูลภาพรหัสแท่ง 2 มิติในลักษณะของภาพถ่ายที่ได้นั้น เกิดความไม่สมบูรณ์ในกรณีที่รูปภาพเกิดการบิดเบี้ยวดังรูปที่ 3.3 ข เราได้ใช้หลักการดังต่อไปนี้



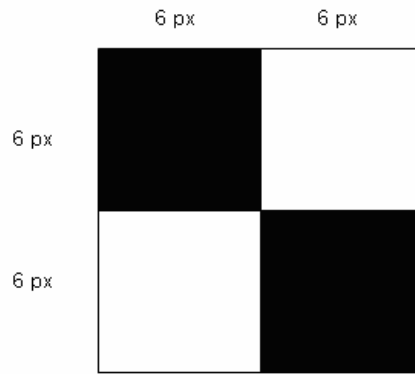
รูปที่ 3.10 ลักษณะของภาพถ่ายที่บิดเบี้ยว

ในการแก้ไขภาพที่บิดเบี้ยวนั้นขั้นตอนแรกคือการหมุนของภาพทั่วทั้ง 4 จุดก่อน โดยหาจุดที่ใกล้กับขอบภาพที่สุด (ซ้ายบน ขวาบน ซ้ายล่าง ขวาล่าง) หลังจากนั้นก็ลากเส้นตามขอบของข้อมูลเพื่อหาระยะห่างจากขอบภาพ เมื่อเราได้ระยะห่างของข้อมูลกับภาพแล้วเราก็เริ่มดึงภาพที่ละแถวจนครบทุกแถว

3.3.3 การเปลี่ยนภาพรหัสแท่ง 2 มิติให้อยู่ในรูปของตัวข้อมูล

หลังจากทำการประมวลผลภาพในขั้นตอนต่างๆ ตั้งแต่การทำ Binarization หรือว่าการทำให้รูปภาพเป็นแค่รูปขาวหรือดำ และขั้นตอนการปรับปรุงภาพที่ไม่สมบูรณ์ให้ภาพกลับมามีความชัดเจนน้อยจนภาพที่ได้มามีความเหมาะสมเพียงพอที่จะนำมาใช้ในการประมวลผลแล้ว

ในขั้นตอนต่อไปคือการแปลงภาพรหัสแท่ง 2 มิติให้อยู่ในรูปของตัวข้อมูลตัวเลขและตัวอักษรโดยมีวิธีการดังต่อไปนี้

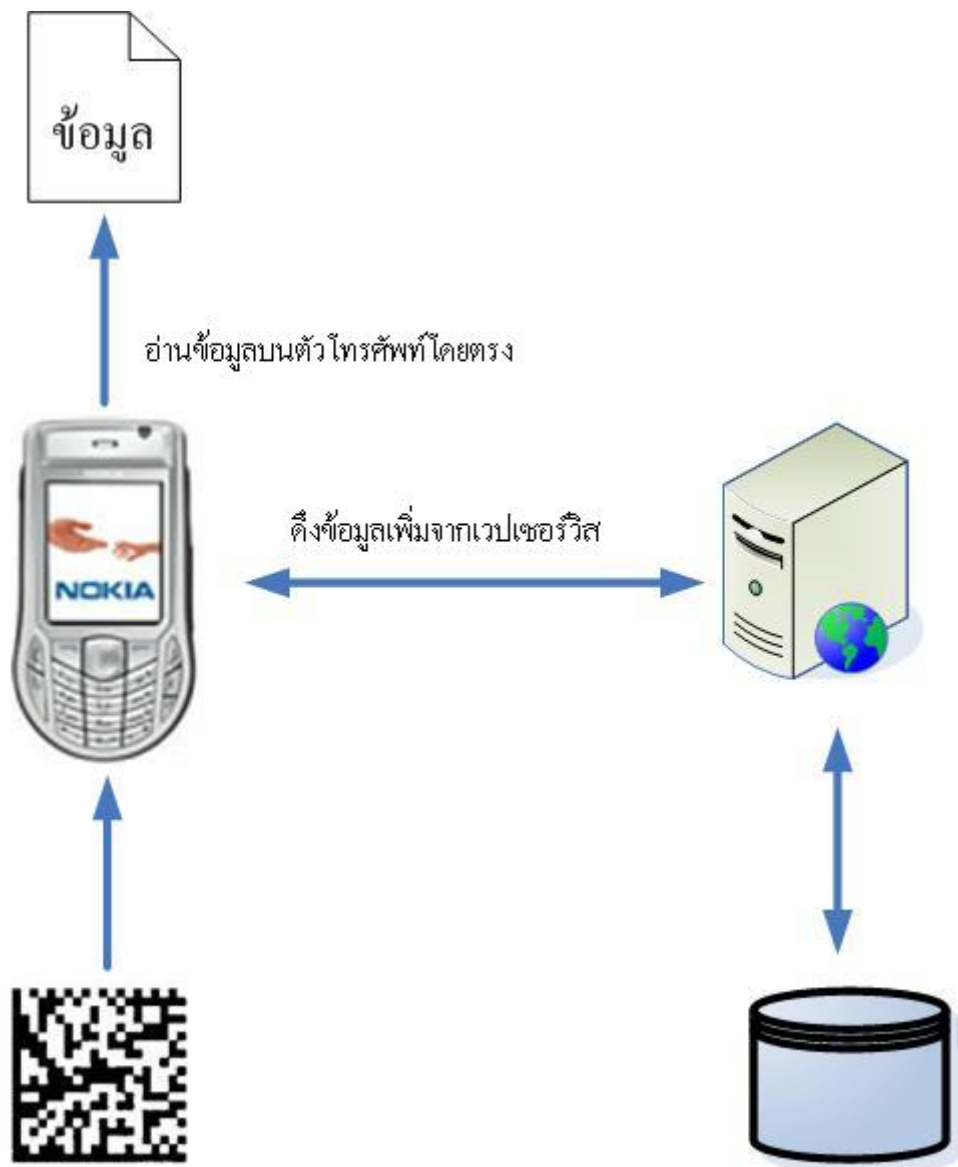


รูปที่ 3.11 การเทียบระดับข้อมูลขาวดำ

การเทียบระดับข้อมูลนั้น เราใช้วิธีการหาอัตราส่วนของแต่ละช่องข้อมูลดังตัวอย่างในรูปที่ 3.11 ถ้าข้อมูลแต่ละช่องมีขนาด 36 พิกเซล แล้วใน 36 พิกเซล นั้นมีค่าระดับสีดำอยู่เกิน 18 พิกเซล เราจะแทนช่องนั้นเป็นสีดำ หรือ 1 นั่นเอง

3.4 การออกแบบในส่วนของการนำมาประยุกต์ใช้งาน

ในการการนำมาประยุกต์ใช้งาน โครงสร้างของระบบการจำแนกหัตถ์แบ่งด้วยการประมวลผลภาพจะแบ่งออกเป็น 2 ส่วนคือ ส่วนของเซิร์ฟเวอร์ (Server) และส่วนของไคลเอนท์ (Client) โดยจะแสดงดังรูปที่ 3.12



รูปที่ 3.12 การนำมาประยุกต์ใช้งาน

1. ส่วนของเซิร์ฟเวอร์นั้นจะทำหน้าที่เป็นฐานข้อมูลรองรับการร้องขอบริการผ่านเว็บเซอวิสจากมือถือ แล้วจะทำการประมวลผลเพื่อตอบสนองการบริการต่างๆ
2. ในส่วนของไคลเอนต์หรือตัวโทรศัพท์มือถือนั้นจะแบ่งรูปแบบในการทำงานออกเป็น 2 ส่วนได้แก่

ส่วนที่ใช้ในการประมวลผลภาพรหัสแท่ง โดยใช้กล้องดิจิตอลบนตัวโทรศัพท์มือถือทำการถ่ายภาพมาเพื่อทำการประมวลผลภาพให้ได้ค่าที่ต้องการ โดยจะใช้หลักการเปลี่ยนข้อมูลในรูปของรหัสแท่งให้อยู่ในรูปแบบตัวเลขและตัวอักษร หรือที่เรียกกันว่าข้อมูลแบบ Text file กล่าวคือจะได้ข้อมูลพื้นฐานที่ใช้ในการอ้างอิงข้อมูลต่อไป

ส่วนการเรียกใช้บริการของเซิร์ฟเวอร์ โดยเรียกผ่านจาวาแอปพลิเคชันไปที่เว็บเซอร์วิส โดยจะทำการส่งค่าที่ใช้ในการอ้างอิงข้อมูลที่ต้องการ หรือ คีย์เวิร์ดต่างๆ ไปให้กับเว็บเซอร์วิส เพื่อให้เซิร์ฟเวอร์ทำการประมวลผลแล้วส่งรายละเอียดของข้อมูลที่ต้องการนั้นๆ กลับมาในเครื่อง โทรศัพท์มือถือ เช่น ชนิด ราคา จำนวน และคุณสมบัติของสินค้า ฯลฯ

3.4.1 การออกแบบระบบในส่วนของโทรศัพท์มือถือ

โปรแกรมบนมือถือนั้นใช้ความเข้าใจในเรื่องการประมวลผลภาพจากข้างต้นมาสร้างเป็น โปรแกรมอ่านภาพรหัสแท่งสองมิติ ชนิดคิวอาร์โค้ด ด้วยจาวาแอปพลิเคชัน โดยมีลักษณะดังนี้



รูปที่3.13 ตัวแอปพลิเคชัน บนโทรศัพท์มือถือ

ตัวมือถือในรูปที่3.13 นั้นเราได้ใช้โปรแกรมจำลองการทำงาน (Emulator) ของมือถือ Nokia Series60 MIDP SDK v1.0 ซึ่งสนับสนุนแพลตฟอร์มของ J2ME มาใช้ในการพัฒนาซอฟต์แวร์ เพื่อให้สะดวกต่อการทดลองในโครงการนี้

เมนูการใช้งานได้แบ่งออกเป็น 3 เมนูย่อยด้วยกัน เมนูแรกคือ **Camera** เป็นการเลือกประมวลผลภาพรหัสแท่งสองมิติ ชนิดคิวอาร์โค้ดจากรหัสภาพที่ต้องการ และอ่านค่ามาแสดงที่หน้าจอมือถือโดยตรง เมนูที่สองคือ **Query** เป็นการส่ง ID ไปค้นหาข้อมูลเพิ่มจากเว็บเซอร์วิสในกรณีที่ต้องการรายละเอียดของข้อมูลเพิ่มเติมจาก ID ของรหัสแท่งนั้นๆ (กรณีที่สองเป็นแนวคิดในการพัฒนาต่อ) เมนูที่สามคือ **Exit** เป็นการยกเลิกและออกจากการใช้งานปัจจุบัน



รูปที่3.14 การแสดงเมนูการใช้งานของแอปพลิเคชัน

หลังจากการถ่ายภาพรหัสสองมิติ ชนิด คาด้าเมทริกซ์ ตัวมือถือจะทำการรับภาพเข้ามา
แสดงบนหน้าจอมือถือดังรูปที่ 3.15



รูปที่3.15 การแสดงข้อมูลภาพบนโทรศัพท์มือถือ

การอ่านข้อมูลรหัสสองมิติ ชนิดคาด้าเมทริกซ์บนตัวมือถือนั้นทำได้โดยการเลือกที่
Options แล้วไปที่เมนู **Camera** ผลที่ได้ก็จะแสดงข้อมูลที่อ่านได้จากตัวรหัสสองมิติ ออกมา
ทางด้านซ้ายล่างของหน้าจอโทรศัพท์มือถือ ตามด้วยข้อความความผิดพลาดที่เกิดขึ้นหลังจากการ
ประมวลผลภาพ



รูปที่3.16 การแสดงข้อมูลหลังจากผ่านการประมวลผล

3.4.2 การออกแบบระบบในส่วนของเว็บเซอร์วิส

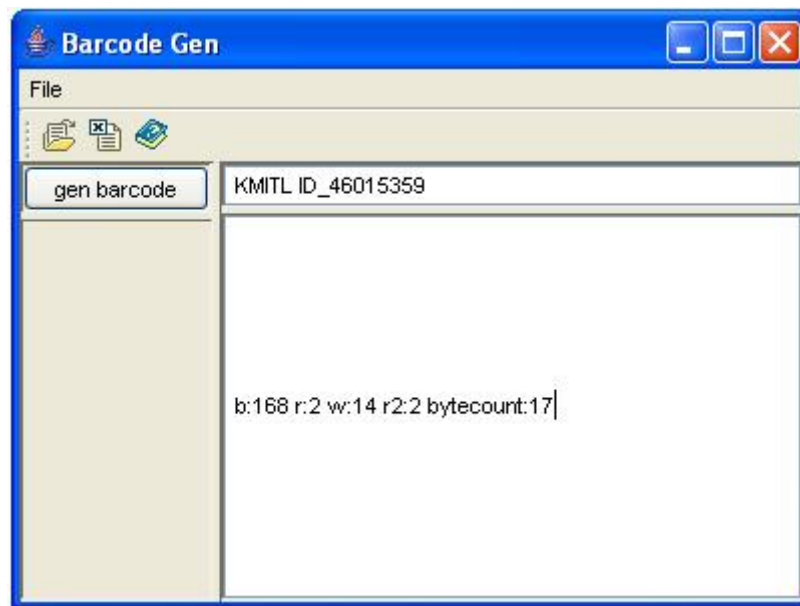
การติดต่อกับเว็บเซอร์วิสนั้นเราจะส่ง ID ของข้อมูลตัวรหัสแท่งสองมิติ ชนิดดาต้าเมทริกซ์ที่อ่านได้ ไปค้นหาข้อมูลเพิ่มจากเว็บเซอร์วิสโดยการเลือก **Option** แล้วไปที่เมนู **Query** ผลที่ได้คือเว็บเซอร์วิสจะส่งข้อมูลรายละเอียดของ ID ที่ต้องการกลับมาแสดงที่หน้าจอโทรศัพท์มือถือ



รูปที่3.17 แสดงผลการเรียกใช้เว็บเซอร์วิสบนโทรศัพท์มือถือ

3.4.3 โปรแกรมสร้างภาพรหัสแท่งสองมิติ ชนิดดาต้าเมทริกซ์

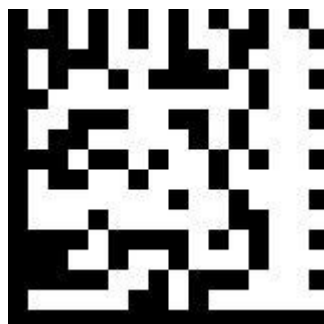
สำหรับตัวสร้างรหัสแท่งสองมิติ นี้เราได้ทำการสร้างมาตรฐานการพิมพ์รหัสแท่งสองมิติ ชนิด ดาต้าเมทริกซ์ ขึ้นมาเองเพื่อให้ตรงกับวิธีการเข้ารหัสและถอดรหัสของการทดลองที่สร้างขึ้นมา เนื่องจากมาตรฐานของดาต้าเมทริกซ์ มีอยู่ด้วยกันหลายมาตรฐานโดยแอปพลิเคชันที่สร้างขึ้นมา มีลักษณะดังรูปที่ 3.18



รูปที่3.18 โปรแกรมสร้างภาพรหัสแท่ง สองมิติ ชนิด คาด้าเมทริกซ์

จากรูปที่3.18 ด้านบนคือช่องสำหรับใส่ข้อมูลที่ต้องการสร้างเป็นรหัสแท่งสองมิติ ชนิด คาด้าเมทริกซ์ในการทดลองเราได้ใส่ข้อมูลเป็น “KMITL ID_46015359” ส่วนด้านล่างคือรายละเอียดหลังจากการเข้ารหัสตัวข้อมูลที่ต้องการแล้วโดย **b:** คือจำนวนบิตข้อมูล, **r:** ค่าความผิดพลาดที่สามารถเกิดขึ้นได้, **w:** คือจำนวนช่องที่ใช้สร้างคาด้าบิต, **bytecount:** คือจำนวนตัวอักษรของข้อมูล

จากนั้นตัวโปรแกรมจะทำการสร้างรหัสแท่งสองมิติ ชนิด คาด้าเมทริกซ์ขึ้นมาดังรูปที่ 3.19 โดยภาพรหัสที่ถูกสร้างขึ้นจะอยู่ในไคลฟ์ชื่อ imageTest.jpeg



รูปที่3.19 ภาพที่ได้จากโปรแกรมโดยมีข้อมูลเป็น “KMITL ID_46015359”